Gestión de dependencias y ciclo de vida del software

Modelos conceptuales y herramientas: Maven, Gradle, npm, Yarn, pip y otros ecosistemas

Mega Plumbing LLC

Desarrollo de Software - ISI - FRRe - UTN
Dr. Jorge E. Villaverde

Mega Plumbing LLC

Propósitos de aprendizaje

Al finalizar, el estudiante podrá explicar qué es una dependencia y modelarla como parte de un grafo.

Podrá diferenciar dependencia directa, transitiva, de desarrollo/test y de runtime.

Podrá describir el rol de un build/package manager dentro del ciclo de vida.

Podrá comparar Maven, Gradle, npm/Yarn y pip sin confundir herramienta, formato de proyecto y repositorio.

Podrá identificar prácticas para mejorar reproducibilidad, seguridad y mantenibilidad.

Pregunta inicial

¿Qué ocurre si mañana una dependencia deja de existir?

¿Podemos reconstruir exactamente el software que construimos hoy?

Disponibilidad

¿De dónde se obtiene el artefacto?

Resolución

¿Qué versión terminó entrando?

Integridad

¿Es el artefacto esperado?

Concepto: dependencia

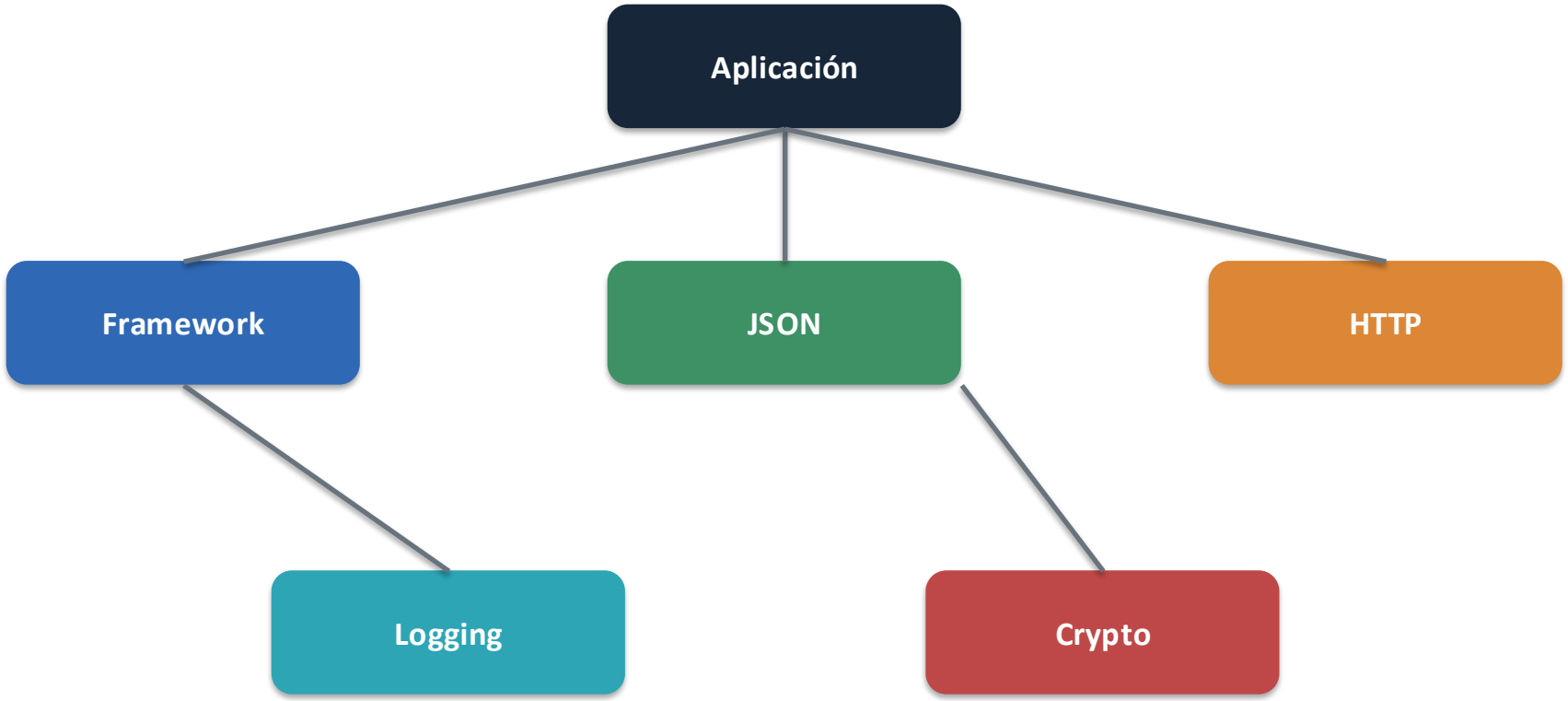
Una dependencia es un artefacto externo requerido para construir, probar, ejecutar o distribuir un sistema.

Puede ser código, plugin, herramienta, runtime, framework o recurso generado.

El sistema mantiene una relación de dependencia respecto de una versión o conjunto de restricciones.

Formalmente podemos pensar el proyecto como un grafo dirigido G = (V, E), donde los vértices son artefactos y las aristas representan “depende de”.

Dependencias transitivas: el grafo crece rápidamente



El gestor de dependencias resuelve este grafo bajo un conjunto de reglas.

Tipos de dependencias

Directa

Declarada explícitamente por el proyecto.

Ej.: aplicación → Jackson.

Transitiva

Llega a través de otra dependencia.

Ej.: aplicación → A → B.

Build / plugin

Necesaria para construir, generar o verificar.

Test / desarrollo

Necesaria durante tests o desarrollo, pero no necesariamente en runtime.

Resolución de versiones

El proyecto puede declarar restricciones, no necesariamente una versión única.

Cuando distintos caminos del grafo solicitan versiones incompatibles, el gestor necesita una estrategia de resolución.

Ejemplos de estrategias: versión más cercana, versión seleccionada por reglas, constraints, overrides o resolución explícita.

Diagnóstico: no mirar solamente el manifest; inspeccionar el grafo resuelto.

```
Aplicación
 ├── A ──> C 1.4
 └── B ──> C 2.1
```

Pregunta: ¿qué versión de C termina en el classpath?

Versionado semántico y rangos

MAJOR

2.0.0

Cambios incompatibles según la convención.

MINOR

1.5.0

Nueva funcionalidad compatible.

PATCH

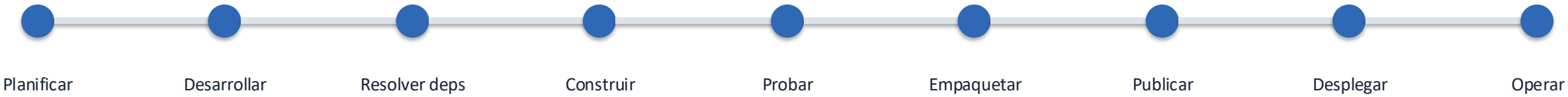
1.5.2

Correcciones compatibles.

SemVer comunica intención; no demuestra que una biblioteca realmente respete compatibilidad.

Un rango más amplio aumenta flexibilidad, pero también el espacio de posibles resoluciones.

Ciclo de vida del software



Un build tool convierte parte de este proceso en un procedimiento automatizado y verificable.

El objetivo no es solamente compilar: es producir artefactos confiables y repetibles.

Tres capas que conviene distinguir

Build / Package Manager

Maven, Gradle, npm, pip…

Resuelve, construye, prueba o instala según el ecosistema.

Repositorio

Maven Central, npm Registry, PyPI, Artifactory, Nexus…

Almacena y distribuye artefactos.

CI/CD

GitHub Actions, GitLab CI, Jenkins…

Ejecuta el proceso automáticamente.

No son sinónimos: una organización puede combinar herramientas de las tres capas.

Maven: modelo de proyecto y lifecycle

pom.xml: metadata, dependencias, plugins y configuración.

Convenciones fuertes: estructura estándar y lifecycles conocidos.

Fases típicas: validate → compile → test → package → verify → install → deploy.

El dependency tree permite observar dependencias directas y transitivas.

```
<dependency>
  <groupId>org.example</groupId>
  <artifactId>library</artifactId>
  <version>1.2.3</version>
</dependency>

mvn test
mvn dependency:tree
```

Gradle: tasks, plugins y build como código

build.gradle / build.gradle.kts define plugins, dependencias, tasks y reglas.

El modelo central es un grafo de tasks: las tareas pueden depender de otras tareas.

Gradle Wrapper fija la versión de Gradle utilizada por el proyecto.

Las configuraciones de dependencias permiten expresar roles como implementation y testImplementation.

```
plugins { id 'java' }

dependencies {
  implementation 'org.example:library:1.2.3'
  testImplementation 'org.junit.jupiter:junit-jupiter:...'
}

./gradlew test
./gradlew dependencyInsight --dependency library
```

npm / Yarn: manifiesto + lockfile

package.json expresa metadata, scripts y dependencias.

dependencies, devDependencies y peerDependencies tienen semánticas distintas.

El lockfile registra una resolución concreta del grafo.

En CI es importante diferenciar instalación flexible de instalación reproducible.

```
{
  "scripts": { "test": "jest" },
  "dependencies": { "express": "^5.0.0" },
  "devDependencies": { "typescript": "..." }
}

npm ci
npm ls
# Yarn: yarn why <package>
```

Python: pip, entornos y metadatos

pip es el instalador de paquetes; el ecosistema actual también utiliza pyproject.toml y herramientas de resolución/gestión de proyectos.

venv permite aislar el entorno de un proyecto.

requirements.txt puede representar un conjunto de versiones instaladas o deseadas.

El objetivo académico es distinguir: formato de metadata, instalador, resolvedor y herramienta de proyecto.

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
pip show requests
pip freeze
```

El patrón se repite en otros ecosistemas

C# / .NET

NuGet
.csproj + packages.lock.json

Rust

Cargo
Cargo.toml + Cargo.lock

Go

Go modules
go.mod + go.sum

PHP

Composer
composer.json +
composer.lock

Manifest vs. lockfile

Manifest

Declara la intención o restricciones del proyecto.

Ej.: package.json, Cargo.toml.

Lockfile

Captura una resolución concreta.

Ej.: package-lock.json, Cargo.lock, yarn.lock.

Repositorio

Proporciona los artefactos que satisfacen esa resolución.
Ej.: registry, proxy, repositorio privado.

Reproducibilidad = controlar entradas, resolución, artefactos y entorno.

Reproducibilidad: una definición operacional

Una build es reproducible cuando, dadas las mismas entradas relevantes, produce resultados equivalentes.

Entradas relevantes: código fuente, dependencias, versiones de herramientas, configuración, plataforma y artefactos externos.

Prácticas: wrapper/toolchain, lockfiles, imágenes de build, repositorios controlados y CI limpio.

Pregunta de auditoría: “¿Podemos reconstruir este artefacto seis meses después?”

Gestión de dependencias y seguridad

Las dependencias amplían la superficie de ataque del sistema.

Una vulnerabilidad puede estar en una dependencia transitiva que el equipo nunca declaró directamente.

Software Bill of Materials (SBOM) permite describir los componentes presentes en un artefacto.

Buenas prácticas: escaneo automatizado, actualización periódica, revisión de advisories y políticas para componentes críticos.

Una alerta de vulnerabilidad no equivale automáticamente a riesgo explotable: debe analizarse el contexto.

Gobernanza de dependencias

Política

¿Qué versiones se permiten?

¿Qué licencias?
¿Qué fuentes?

Proceso

Actualizar → probar → revisar → aprobar → desplegar.

Automatización

Dependabot/Renovate u otras herramientas + CI.

Excepciones

Registrar por qué una dependencia vulnerable o antigua permanece.

Problemas frecuentes y cómo diagnosticarlos

“Funciona en mi máquin

JDK/Node / Python distintos → wrapper, toolchain, versiones explícitas.

Conflicto de versiones

Dos caminos traen la misma librería.
→ dependency tree / why / graph.

Build lento

Descargas y tareas repetidas → caches, daemon, build incremental.

CI diferente

Lockfile / repo / credenciales cambian → entorno reproducible.

Diagnóstico: del síntoma al grafo

Síntoma: “en mi máquina funciona”.

Paso 1: comparar versión del runtime / build tool.

Paso 2: inspeccionar dependencias resueltas.

Paso 3: identificar quién introduce la versión conflictiva.

Paso 4: verificar repositorio, caché y lockfile.

Paso 5: reproducir en entorno limpio.

Paso 6: fijar la solución y agregar una protección en CI.

Actividad: resolver un conflicto de dependencias

A → C 1.4
B → C 2.1
Aplicación → A + B

El test falla con NoSuchMethodError.

Preguntas:
1. ¿Qué evidencia buscarían?
2. ¿Qué comando usarían?
3. ¿Qué alternativas de solución existen?
4. ¿Cómo evitarían la regresión?

Dinámica: 5 min en parejas + 3 min de puesta en común.

Comparación conceptual

Maven

Lifecycle + POM + plugins
Fuerte convención.

Gradle

Tasks + plugins + DSL
Alta expresividad.

npm / Yarn

package.json + lockfile
Scripts + registry.

pip + ecosistema Python

Instalación + entornos + metadata
Varias herramientas complementarias.

La comparación útil no es “cuál es mejor”, sino qué problema resuelve cada mecanismo.

Checklist para proyectos reales

- ✓ Usar wrapper / toolchain para fijar la versión del build tool cuando aplique.
- ✓ Versionar manifests y lockfiles cuando corresponda.
- ✓ Fijar versiones de herramientas de build.
- ✓ Inspeccionar dependencias transitivas.
- ✓ Automatizar tests y análisis de dependencias en CI.
- ✓ Usar repositorios/proxies confiables.
- ✓ Actualizar periódicamente y revisar breaking changes.
- ✓ Generar SBOM cuando el contexto lo requiera.
- ✓ Documentar cómo reproducir una build desde cero.
- ✓ Mantener manifests y lockfiles bajo control de versiones cuando corresponda.
- ✓ Evitar versiones “flotantes” en producción.

Preguntas

Antes de Maven/Gradle: “¿Qué cosas necesitan descargar hoy para compilar un proyecto?”

Antes del grafo: “Si A depende de B y B depende de C, ¿quién declaró C?”

Antes del lockfile: “Dos desarrolladores hacen npm install el mismo día: ¿tienen garantizado el mismo árbol?”

Antes de seguridad: “¿Cómo sabemos qué componentes contiene el binario que entregamos?”

Durante el ejercicio: pedir que justifiquen la solución con evidencia del grafo, no con intuición.

Ideas fundamentales

1

Una dependencia es parte de las entradas del software, no un detalle accesorio.

2

El gestor resuelve un grafo y automatiza etapas del ciclo de vida.

3

La reproducibilidad requiere controlar versiones, resolución, artefactos y entorno.

Preguntas / discusión

Referencias y documentación

Maven Project — Introduction to the Build Lifecycle / Dependency Management.

Gradle Documentation — Dependency Management / Build Lifecycle / Gradle Wrapper.

npm Documentation — package.json / package-lock.json / npm ci.

Yarn Documentation — dependency management and lockfiles.

Python Packaging User Guide — pyproject.toml, virtual environments and packaging.

Semantic Versioning 2.0.0 — semver.org.

OWASP — Software Component Verification Standard (SCVS) y Dependency-Check ecosystem.